

ClickAndGo

Evaluacion Sumativa N2 - Unidades 2 y 3

Software orientado a objetos, persistencia y seguridad en Python

Documento tecnico de la solucion

Institucion	Instituto Profesional INACAP - Sede Puente Alto
Carrera	Ingenieria Informatica
Asignatura	Programacion Orientada a Objeto Seguro - TI3V21
Seccion	114-2A-F2 - Primavera 2026
Negocio asignado	05 - Tienda de e-commerce: ClickAndGo
Integrantes	Miguel Troncoso Alexandy Remicinthe
Docente	Michael Arjel
Fecha de entrega	Miercoles 7 de octubre de 2026, 19:50 hrs
Repositorio	https://github.com/MiguelTroncoso/ecommerce
Demostracion en linea	https://inacap.superflash.site

1. Resumen ejecutivo

ClickAndGo es una tienda de comercio electronico que vende, dentro de un mismo pedido, productos fisicos, productos digitales, servicios tecnicos agendados y electronica importada cotizada en dolares. El sistema desarrollado en Python resuelve ese escenario con un modelo de clases orientado a objetos, persistencia real en SQLite, validacion obligatoria de los datos del cliente, dos reglas de negocio que impiden operaciones inseguras y un precio que depende de un indicador externo obtenido desde una API publica.

La solucion se organiza en cuatro capas (modelo, acceso a datos, servicios e interfaz de consola) y cumple los seis requisitos minimos de la ficha del negocio exigidos por la rubrica. Adicionalmente se publico una demostracion web con una consola en vivo que ejecuta el mismo programa del repositorio.

2. El problema y la solucion propuesta

Una tienda que vende tipos de producto con mecanicas de entrega distintas enfrenta problemas concretos: el flete solo aplica a bienes tangibles, la entrega digital no puede depender de un despacho, un servicio debe quedar agendado para una fecha futura, y el precio de la electronica importada cambia todos los dias con el dolar. Ademias, la operacion necesita controles: no se puede vender sin stock ni entregar mercaderia que no este pagada, y solo algunos roles pueden modificar el catalogo.

La solucion aplica los cuatro pilares de la programacion orientada a objetos. La abstraccion define el contrato de un producto; la herencia especializa ese contrato en subtipos; el polimorfismo permite que cada subtipo resuelva su propia entrega; y el encapsulamiento protege el estado interno con atributos privados y propiedades validadas.

3. Arquitectura del sistema

El proyecto se divide en cuatro carpetas con responsabilidades separadas. Esta separacion permite cambiar la base de datos o la interfaz sin tocar las reglas del negocio.

Capa	Carpeta	Responsabilidad
Modelo	model/	Clases del diagrama UML: cliente, productos, pedido, detalle y trabajadores.
Persistencia	dao/	CRUD en SQLite con consultas parametrizadas y transacciones atomicas.
Servicios	servicios/	Validacion de entradas, consumo de la API del dolar e inicializacion.
Interfaz	main.py	Menu de consola que orquesta el sistema y captura los errores.
Pruebas	tests/	Guion de pruebas P01 a P19 automatizado con unittest.

4. Modelo de clases orientado a objetos

El modelo esta compuesto por doce clases, cada una en su propio archivo dentro de la carpeta model/. Los subtipos heredan con `super().__init__()` y sobrescriben el metodo que varia; los atributos son privados y se acceden mediante `property` con validacion en el setter.

Clase	Tipo	Responsabilidad
Cliente	Entidad	Identidad del comprador; valida RUT (Modulo 11) o correo.
Producto	Abstracta	Contrato del catalogo: id, nombre, precio, stock, entrega y precio final.

Clase	Tipo	Responsabilidad
ProductoFisico	Subtipo	Despacho por transporte; calcula el flete por peso, volumen y distancia.
ProductoDigital	Subtipo	Entrega inmediata con enlace y licencia; flete cero.
ProductoServicio	Subtipo	Agenda la visita tecnica para una fecha y hora futura.
ProductoElectronica	Subtipo	Hereda de ProductoFisico y cotiza en USD con el dolar del dia.
DetallePedido	Composicion	Linea del pedido; congela cantidad y precio unitario pactado.
Pedido	Transaccion	Orquesta pago, despacho, totales y las dos reglas de negocio.
Trabajador	Abstracta	Personal, autenticacion (PBKDF2) y permisos por rol.
EncargadoBodega	Subtipo	Puede despachar; no puede modificar el catalogo.
Administrador	Subtipo	Control total del catalogo y de los precios.
Excepciones	Apoyo	Una excepcion propia por cada regla que impide una operacion.

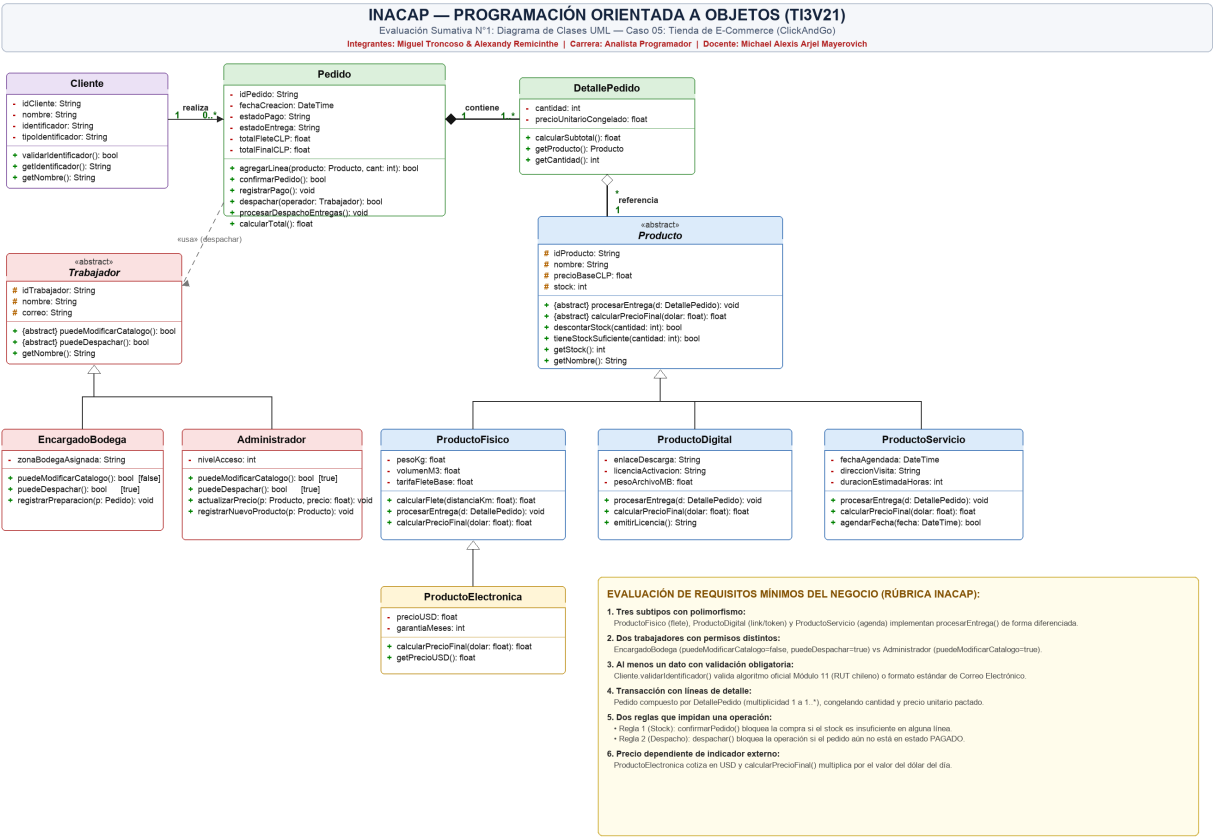


Figura 1. Diagrama de clases UML del caso ClickAndGo.

5. Persistencia de datos

La base de datos es SQLite y se crea automaticamente al iniciar el programa: no requiere instalacion ni scripts manuales. La capa dao/ esta completamente separada del modelo, de modo que ninguna clase del modelo contiene SQL.

Tabla	Contenido	Relacion
trabajadores	Usuarios, rol, hash PBKDF2 y salt	-

Tabla	Contenido	Relacion
clientes	Nombre e identificador validado (RUT o correo)	1 a 0..* pedidos
productos	Los cuatro subtipos de producto en una sola tabla con columna tipo	1 a * detalles
pedidos	Cabecera: cliente, fecha, estados y totales	1 a 1..* detalles
detalle_pedido	Lineas con cantidad y precio unitario congelado	Clave foranea a pedidos
configuracion	Valores de apoyo del sistema	-

5.1 CRUD completo

Las entidades principales tienen operaciones de crear, listar, obtener, actualizar y eliminar implementadas en DAO separados. El catalogo se reconstruye polimorficamente: el ProductoDAO revisa la columna tipo y devuelve la instancia de la clase correcta.

```
ProductoDAO.fila_a_producto(fila) -> ProductoFisico | ProductoElectronica | ProductoDigital |
ProductoServicio
```

5.2 Transaccion con lineas de detalle

El pedido y sus lineas se guardan dentro de una unica transaccion SQL. Si falla la insercion de cualquier linea, no queda un pedido incompleto en la base de datos.

```
with self._bd.conectar() as conexion:
    conexion.execute('INSERT INTO pedidos (...) VALUES (:id_pedido, ...)', pedido.to_dict())
    for detalle in pedido.detalles:
        conexion.execute('INSERT INTO detalle_pedido (...) VALUES (...) ', {...})
    conexion.commit()
```

5.3 Consultas parametrizadas

Todas las consultas usan parametros enlazados. El texto que escribe el usuario nunca se concatena dentro de una sentencia SQL, lo que neutraliza la inyeccion SQL.

```
SELECT * FROM clientes WHERE id_cliente = :id <- correcto (parametrizado)
'SELECT * FROM clientes WHERE id_cliente = ' + id <- prohibido en este proyecto
```

6. Reglas de negocio implementadas como excepciones propias

Regla	Excepcion	Donde se lanza	Efecto
No confirmar un pedido sin stock suficiente	StockInsuficienteError	Pedido.confirmar_pedido()	Impide la operacion; el stock no cambia
No despachar un pedido impago	PedidoNoPagadoError	Pedido.despachar()	Impide el despacho y mantiene el estado
Solo el rol autorizado modifica el catalogo	SinPermisoError	AplicacionClickAndGo._validar_permiso_catalogo()	El bodeguero no puede crear, modificar ni eliminar
Dato con formato o rango invalido	DatoInvalidoError / IdentificadorInvalidoError	Setters del modelo y validador	Se rechaza el dato y se vuelve a preguntar
La API externa no responde	IndicadorNoDisponibleError	ServicioIndicadorDolar.obtener_valor_dolar()	Se informa y el sistema continua

Cada excepcion se captura de forma especifica en main.py. Un error de negocio muestra un mensaje al operador y devuelve el control al menu: el programa nunca se detiene por un dato invalido ni por una operacion no permitida.

7. Validacion de entradas

Todas las entradas del usuario se validan antes de usarse, considerando tipo, formato y rango. Si el dato es invalido, se informa el problema y se solicita nuevamente.

Dato	Validacion aplicada	Caso de prueba
RUT del cliente	Algoritmo Modulo 11 sobre el cuerpo y digito verificador	P07 / P08
Correo del cliente	Expresion regular de formato sintactico	P07 / P08
Cantidad de una linea	Entero mayor que cero	P19
Precio y stock	Decimal no negativo / entero no negativo	P19
Fecha del servicio	Formato AAAA-MM-DD HH:MM y fecha futura	P11
Enlace de descarga	Debe comenzar con http:// o https://	P10
Opcion de menu	Debe existir entre las opciones validas	P18

El dato obligatorio que exige la ficha (RUT o correo) se valida dentro del setter del modelo, de modo que la proteccion no depende de la interfaz: aunque otro programa use estas clases, el objeto con un identificador invalido no puede existir.

8. Consumo de un servicio externo

El precio de la electronica importada se obtiene multiplicando su valor en dolares por el valor del dolar observado del dia. Ese indicador se consulta a la API publica mindicador.cl mediante la libreria oficial requests, con un tiempo maximo de espera de cinco segundos.

```
respuesta = requests.get(self._url, timeout=self._timeout)
respuesta.raise_for_status()
serie = respuesta.json().get('serie') or []
ultimo = serie[0]
return ValorDolar(float(ultimo['valor']), str(ultimo['fecha'][:10], 'mindicador.cl'))
```

Si la API no responde, el servicio lanza IndicadorNoDisponibleError y guarda el ultimo valor conocido en datos/dolar_respaldo.json. En la siguiente consulta sin conexion el programa utiliza ese respaldo, informa que no pudo obtener el valor del dia y continua funcionando: es exactamente lo que exige el caso P17 del guion de pruebas.

9. Decisiones de seguridad

9.1 Autenticacion de trabajadores

Las contraseñas no se guardan en texto plano. Cada trabajador tiene un salt aleatorio de 16 bytes y su contraseña se almacena como PBKDF2-HMAC-SHA256 con 120.000 iteraciones. La comparacion se realiza con hmac.compare_digest, que evita filtrar informacion por el tiempo de respuesta.

9.2 Control de acceso basado en roles (RBAC)

El permiso no se resuelve con una condicion sobre un texto, sino con el metodo polimorfico de cada trabajador. El EncargadoBodega puede despachar pero no modificar el catalogo; el Administrador puede hacer ambas cosas. Cuando la accion no esta autorizada se lanza SinPermisoError.

9.3 Inyeccion SQL

Todas las sentencias son parametrizadas y la conexion activa PRAGMA foreign_keys = ON para resguardar la integridad referencial. La combinacion de parametros enlazados y validacion previa de tipos cierra las dos vias classicas de ataque sobre una base de datos.

10. Uso de herramientas de IA

La asignatura permite usar asistentes de inteligencia artificial, exigiendo documentar que se adopto, que se modifico y que se descarto, con su razon tecnica.

Sugerencia de la IA	Decision	Razon tecnica
Usar sqlite3.Row y acceder a las columnas por nombre en lugar de por indice numerico.	Adoptada	El codigo queda autoexplicativo y un cambio en el orden del SELECT no rompe la construccion de objetos.
Consumir la API del dolar con requests.get(URL) sin tiempo maximo de espera.	Modificada	Se agrego timeout=5 y captura especifica de fallas de red y de respuesta, porque el criterio 3.1.3 exige continuidad del sistema.
Resolver el tipo de producto con una cadena if/elif dentro de Pedido.	Descartada	La rubrica indica que reemplazar el polimorfismo por if sobre el tipo corresponde al nivel En desarrollo.
Guardar la contrasena con hashlib.md5.	Descartada	MD5 no es apto para contrasenas; se uso PBKDF2-HMAC-SHA256 con salt aleatorio.
Capturar los errores de la API con 'except Exception: pass'.	Descartada	Ocultar el error impide informar al usuario; se capturan excepciones concretas y se informa.

11. Guion de pruebas y resultados

Los 19 casos del guion de pruebas entregado se ejecutan de forma automatizada con el comando python tests/test_guion_pruebas.py. Todos los casos finalizaron satisfactoriamente.

N	Que se prueba	Resultado esperado	Estado
P01	El programa se ejecuta y muestra el menu	El programa inicia y muestra el menu	Cumple
P02	Crear producto	El programa confirma que se guardo	Cumple
P03	Listar producto	Aparece el registro recién creado	Cumple
P04	Modificar producto	El listado muestra el cambio de stock	Cumple
P05	Persistencia al reiniciar	El registro sigue guardado	Cumple
P06	Eliminar producto	El registro ya no aparece	Cumple
P07	Correo o RUT correcto	Lo acepta	Cumple
P08	Correo incompleto	Lo rechaza con mensaje y continua	Cumple
P09	Producto fisico	Despacha por transporte y calcula flete	Cumple
P10	Producto digital	Entrega instantanea con link, sin despacho	Cumple

N	Que se prueba	Resultado esperado	Estado
P11	Servicio	Se agenda para una fecha futura	Cumple
P12	Pedido multi-linea	Queda guardado como un solo registro	Cumple
P13	Detalle del pedido	Se ven todas sus lineas de detalle	Cumple
P14	Confirmar sin stock	Se impide; el stock no cambia	Cumple
P15	Despachar un pedido impago	La operacion se impide	Cumple
P16	Dolar del dia	Usa el valor del dolar de hoy	Cumple
P17	Sin internet	Avisa y no se cae	Cumple
P18	Opcion de menu inexistente	Muestra mensaje y vuelve al menu	Cumple
P19	Letras en un campo numerico	Muestra mensaje y no se cae	Cumple

12. Matriz de cumplimiento de la rubrica

Criterio	Exigencia	Evidencia en el proyecto	Nivel
2.1.2 Modelo orientado a objetos	Clases coherentes con el diagrama, herencia con super() y polimorfismo, atributos privados con property y validacion.	model/ con doce clases, herencia de Producto y Trabajador, propiedades validadas y procesar_entrega() sobrescrito.	Destacado
2.1.3 Librerias oficiales para base de datos	CRUD completo mediante DAO, transaccion con lineas, consultas parametrizadas y tablas creadas al iniciar.	dao/ con cuatro DAO, insercion atomica de pedido y detalles, sentencias con parametros y esquema automatico.	Destacado
2.1.4 Manejo de errores con excepciones propias	Una excepcion propia por cada regla, lanzada y capturada con try/except especifico.	StockInsuficienteError y PedidoNoPagadoError (mas SinPermisoError y otras) capturadas en main.py sin detener el programa.	Destacado
3.1.2 Validacion de entradas	Todas las entradas se validan antes de usarse (tipo, formato y rango).	Setters del modelo y servicios/validador.py con reintento; el dato de la ficha se valida en el setter.	Destacado
3.1.1 / 3.1.3 API externa y continuidad	Precio con indicador obtenido con requests, tiempo maximo de espera y continuidad ante fallas.	ServicioIndicadorDolar con timeout=5, respaldo local y aviso al usuario sin detener el sistema.	Destacado
3.1.4 Seguridad del codigo con apoyo de IA	README que permita instalar y ejecutar, explique decisiones de seguridad y ejemplos concretos de uso de IA.	README con instalacion, decisiones de seguridad y tres casos concretos (adoptado, modificado, descartado).	Destacado

13. Conclusiones

El sistema ClickAndGo cumple los seis requisitos minimos de la ficha del negocio y los seis criterios de la rubrica de la Evaluacion Sumativa N2. La separacion entre modelo, persistencia y servicios permite que las reglas del negocio esten protegidas de forma independiente de la interfaz: el modelo rechaza datos invalidos por si mismo, el DAO impide la inyeccion SQL y la capa de servicios aísla la dependencia de la API externa.

El uso de excepciones propias para cada regla transforma los controles del negocio en parte del diseño del software, y no en simples mensajes en pantalla. Ese es el aporte central del proyecto: un sistema que no se cae cuando el usuario se equivoca ni cuando un servicio externo falla.